Smart Canteen Order and Stock Management

Python Programming – Activity-Based Learning (ABL) Assignment

1. Problem Statement

Develop a Python program to process successive customer orders, calculate bills, update available inventory, reject unavailable quantities, and summarize daily sales and remaining stock.

2. Objectives

• Process multiple orders one after another. • Calculate item totals and order bills. • Check stock before accepting a quantity. • Update inventory only for accepted items. • Display daily sales, rejected items, and remaining stock.

3. Assumptions

The canteen uses a fixed menu and fixed prices. Each order contains item names and quantities. An item is accepted only if its quantity is positive and does not exceed current stock. Rejected items do not affect the bill or inventory. Currency is shown in Indian rupees (Rs.).

4. Algorithm

```
1. Initialize menu items with price and available stock.
2. Set total daily sales and counters to zero.
3. Read each customer order in sequence.
4. For each item, check whether it exists and quantity is valid.
5. If stock is insufficient, reject the item without changing stock.
6. Otherwise calculate cost, subtract stock, and add cost to bill.
7. Add accepted order bill to daily sales.
8. Print sales summary and remaining stock.
```

5. Flowchart (Text Representation)

```
START
  |
Initialize menu, prices, and stock
  |
Read next customer order
  |
Check each item and requested quantity
  |
Stock sufficient and quantity valid?
  | Yes                          | No
Calculate cost; update stock    Reject item; show message
  |                              |
Add item cost to bill <---------+
  |
More items/orders? ---- Yes --> Continue processing
  |
 No
  v
Display daily sales and remaining stock
  |
 END
```

6. Python Source Code

```
# Smart Canteen Order and Stock Management
# Handles successive orders, billing, stock validation, and daily summaries.

menu = {
    "Idli": {"price": 30, "stock": 20},
    "Dosa": {"price": 50, "stock": 15},
    "Meals": {"price": 80, "stock": 10},
    "Tea": {"price": 15, "stock": 30},
}

orders = [
    [("Idli", 2), ("Tea", 2)],
    [("Dosa", 1), ("Meals", 1)],
    [("Tea", 40)],          # Rejected: quantity exceeds available stock
    [("Idli", 3), ("Dosa", 2)],
]

daily_sales = 0
successful_orders = 0
rejected_items = 0
```

```
print("SMART CANTEEN ORDER AND STOCK MANAGEMENT")
print("=" * 78)
print("Initial stock:", ", ".join(f"{item}={data['stock']}" for item, data in menu.items()))
print("-" * 78)

for order_no, items in enumerate(orders, start=1):
    print(f"\nOrder #{order_no}")
    bill = 0
    accepted_any = False

    for item_name, quantity in items:
        if item_name not in menu:
            print(f"  REJECTED: {item_name} is not on the menu.")
            rejected_items += 1
            continue

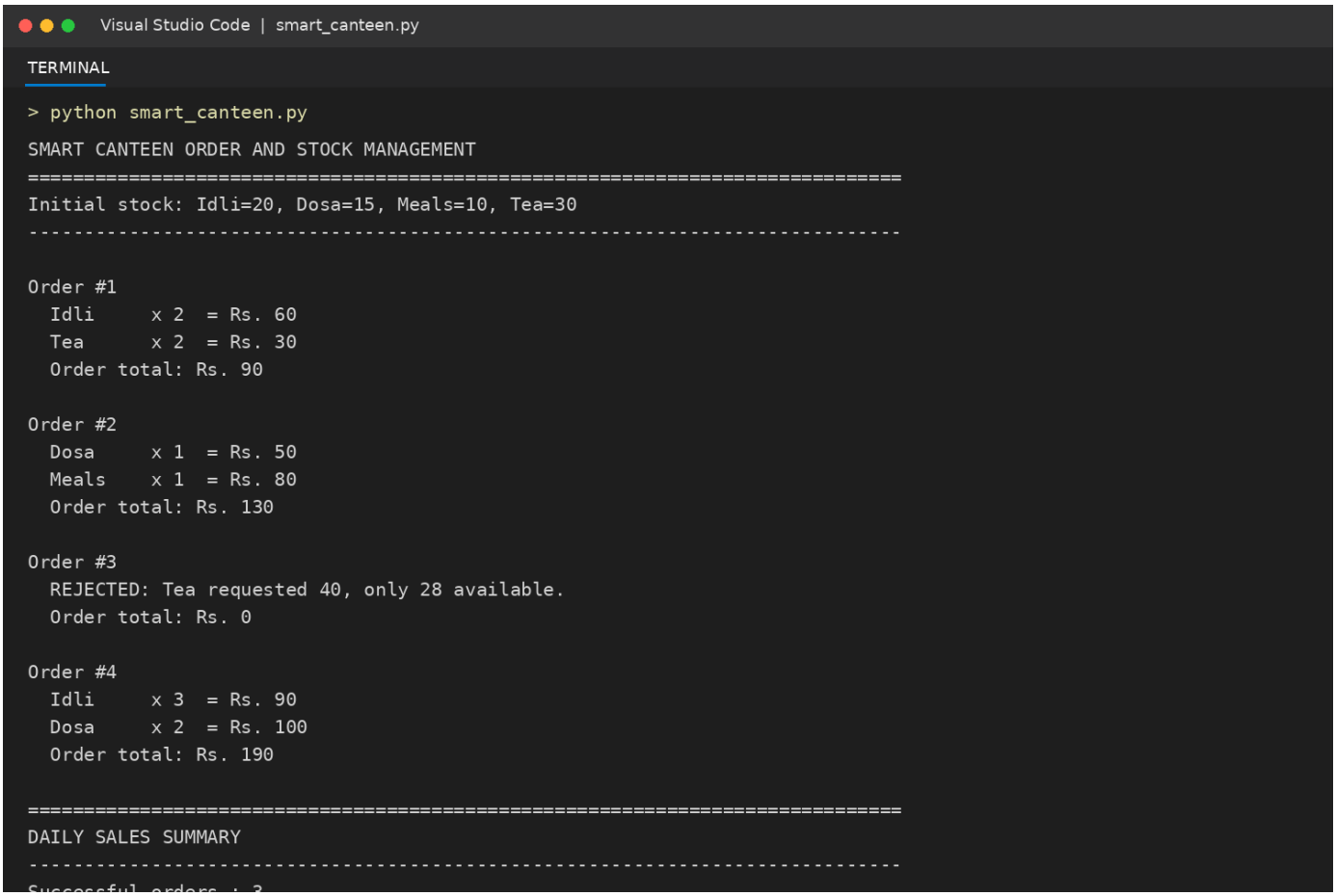
        available = menu[item_name]["stock"]
        if quantity <= 0:
            print(f"  REJECTED: Invalid quantity {quantity} for {item_name}.")
            rejected_items += 1
        elif quantity > available:
            print(f"  REJECTED: {item_name} requested {quantity}, only {available} available.")
            rejected_items += 1
        else:
            cost = menu[item_name]["price"] * quantity
            menu[item_name]["stock"] -= quantity
            bill += cost
            accepted_any = True
            print(f"  {item_name:<8} x {quantity:<2} = Rs. {cost}")

    if accepted_any:
        successful_orders += 1
        daily_sales += bill
    print(f"  Order total: Rs. {bill}")

print("\n" + "=" * 78)
print("DAILY SALES SUMMARY")
print("-" * 78)
print(f"Successful orders : {successful_orders}")
print(f"Rejected items    : {rejected_items}")
print(f"Total sales       : Rs. {daily_sales}")
print("\nREMAINING STOCK")
for item_name, data in menu.items():
    print(f"{item_name:<8}: {data['stock']} units")
print("-" * 78)
print("Result: Orders processed and inventory updated successfully.")
```

7. Sample Execution Output Screenshot

This terminal-style screenshot presents the sample output. Run smart_canteen.py in Visual Studio Code to reproduce the results.



8. Test Cases

```
Test 1: Order 2 Idli and 2 Tea.
Expected: Rs. 90 bill; stock decreases by 2 Idli and 2 Tea.

Test 2: Order 1 Dosa and 1 Meals.
Expected: Rs. 130 bill; stock decreases correctly.

Test 3: Request 40 Tea when only 28 remain.
Expected: Request rejected; stock and bill unchanged for this item.

Test 4: Order 3 Idli and 2 Dosa.
Expected: Rs. 190 bill; stock updates.

Final expected daily sales: Rs. 410.
Remaining stock: Idli=15, Dosa=12, Meals=9, Tea=28.
```

9. Conclusion

The program processes multiple canteen orders, calculates bills, rejects requests that exceed available stock, updates inventory for accepted items, and reports daily sales and remaining stock.